

Smart Attendance App

Complete System Architecture & Technical Design Document

Face identity verification · on-device liveness (mesh + parallax) · geofenced location · time-window enforcement

Prepared July 2026

1. Executive summary

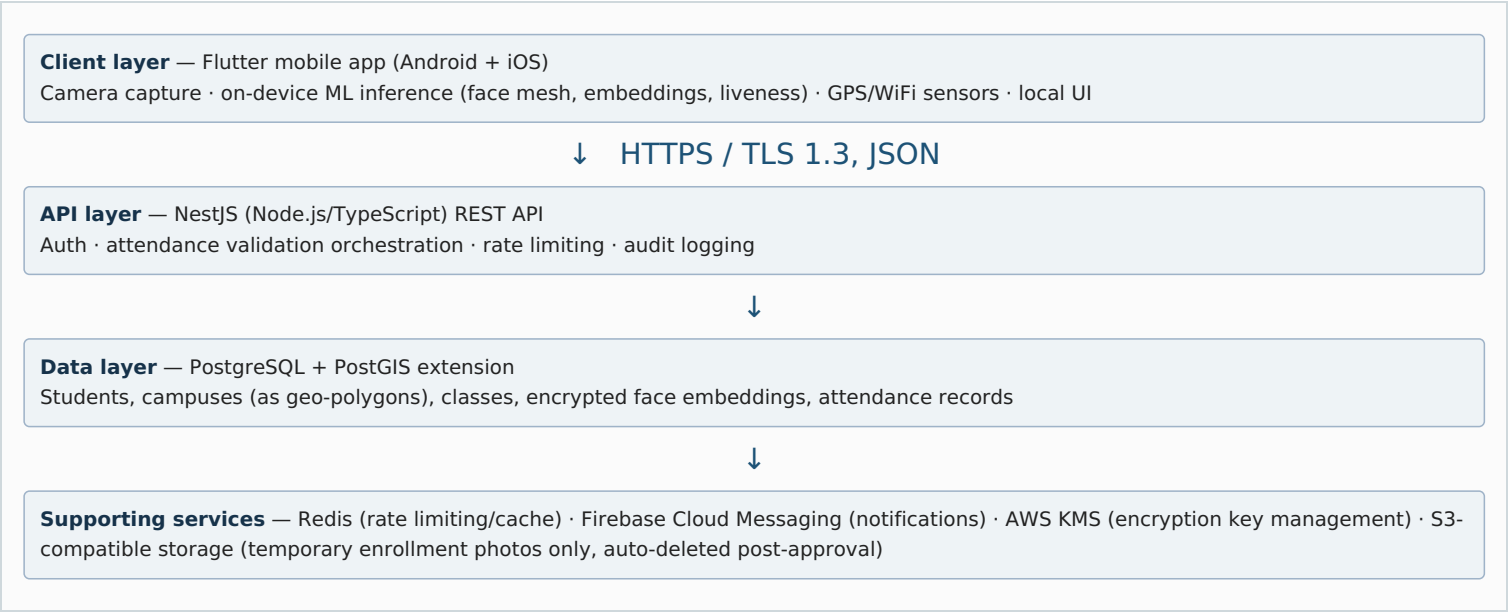
This document specifies the full architecture for a mobile attendance system that verifies a student is (a) physically present, (b) a real live human and not a photo/video/mask, and (c) actually on campus, before marking attendance. It replaces manual roll-call with a verification pipeline that layers three independent checks, so that defeating the system requires defeating all three simultaneously rather than any single one.

Core design principle: this is a **1:1 verification** system, not a 1:N face search. The student identifies themselves first (login), and the camera only answers one question: "does this live face match the face on file for this specific student ID?" This is far more accurate than trying to recognize an unknown face out of an entire student database, and it fails safely — a mismatch simply rejects, it never silently assigns attendance to the wrong person.

1.1 The three verification layers

Layer	Answers the question	Primary technique
Identity	Is this the enrolled student?	On-device face embedding comparison (MobileFaceNet)
Liveness	Is this a real, live 3D face right now?	Active head-turn challenge + MediaPipe Face Mesh (468-pt 3D landmarks) + motion-parallax depth check
Location & time	Are they on campus, during class hours?	GPS polygon check + WiFi BSSID match + server-side time window

2. System architecture



Note: all face recognition and liveness inference runs **on-device**, not via a cloud ML API. The backend never receives raw photos during attendance marking — only a numerical embedding vector and metadata. This keeps the system free to operate at scale (no per-verification API cost) and reduces the amount of sensitive biometric data that ever leaves the student's phone.

3. Technology stack

Layer	Choice	Why this over alternatives
Mobile framework	Flutter	Single codebase for Android/iOS; mature <code>google_mlkit_face_detection</code> , <code>camera</code> , and TFLite plugin ecosystem; compiles to native ARM for fast on-device ML
Face landmarks / head pose	Google ML Kit Face Detection	Free, on-device, gives Euler angles (yaw/pitch/roll) and eye-open probabilities directly — no custom model needed
3D face mesh	MediaPipe Face Mesh (468 points)	Adds estimated depth (x, y, z) per landmark for richer shape analysis and the parallax check
Face identity embeddings	MobileFaceNet (TFLite, ~5 MB)	Purpose-built for on-device face recognition; ~30ms inference on mid-range phones; open weights
Backend	NestJS (Node.js/TypeScript)	Typed, modular, good fit for a clearly separated auth/attendance/admin API

framework		
Database	PostgreSQL + PostGIS	Native geospatial functions (<code>ST_Contains</code> , <code>ST_DWithin</code>) for accurate polygon-based geofencing, not hand-rolled distance math
Cache / rate limiting	Redis	Fast per-device rate limiting and schedule caching
Push notifications	Firebase Cloud Messaging	Free tier covers most institutions; notifies students when an attendance window opens
Hosting (MVP)	Supabase (Postgres + Auth + Storage bundled)	Fastest path to a working pilot; migrate to AWS ECS/RDS if scaling beyond one institution
Encryption key management	AWS KMS or equivalent	Manages the AES-256 keys used to encrypt stored face embeddings at rest

4. Verification pipeline (end-to-end flow)



Any failed step rejects the attempt and returns a specific reason to the app (not a generic error) — this matters both for user experience (student knows exactly what to fix) and for the audit trail (staff can see why a record was flagged).

5. Face enrollment

Enrollment quality determines matching accuracy for the entire semester, so it gets extra rigor:

- Capture **5 photos** at enrollment: front-facing, slight left, slight right, and with/without glasses if applicable.
- Run each through ML Kit face detection — reject if no face found, multiple faces found, face occupies <30% of frame, or lighting is outside an acceptable brightness range.
- Generate embeddings for all 5 and **average them** into one reference embedding, which makes matching robust to small angle/lighting variation instead of depending on a single snapshot.
- Encrypt the final embedding (AES-256, KMS-managed key) before storing — raw enrollment photos are deleted after step 5.
- Require **staff/admin approval** of enrollment before it goes live. This closes the biggest loophole in any biometric system: someone enrolling under the wrong identity in the first place.

6. Liveness detection — full technical detail

6.1 Active challenge-response

A randomized action is requested each attempt (never the same one twice in a row) so a pre-recorded clip can't be reused: `turn_right` , `turn_left` , or `blink` .

```
// Flutter + google_mlkit_face_detection
final actions = ['turn_right', 'turn_left', 'blink'];
final challenge = actions[Random().nextInt(actions.length)];
```

```
void onFrame(Face face) {
    if (challenge == 'turn_right' && face.headEulerAngleY! < -20) passed = true;
    if (challenge == 'turn_left' && face.headEulerAngleY! > 20) passed = true;
    if (challenge == 'blink' &&
        face.leftEyeOpenProbability! < 0.2 &&
        face.rightEyeOpenProbability! < 0.2) passed = true;
}
```

Calculated thresholds: yaw angle $\geq 20^\circ$ (below this is too easy to fake by tilting the phone rather than turning the head; above 30° is uncomfortable for the user). Time window: 4 seconds from prompt to completion. The identity match (Section 7) is run on the exact frame where the challenge completes, not a separate frame, so a face swap mid-challenge is caught.

6.2 3D face mesh (MediaPipe Face Mesh)

468 facial landmarks are extracted with estimated (x, y, z) coordinates per point. This gives real facial *shape* — nose bridge height, cheekbone curvature, eye socket depth — rather than a flat 2D outline. This is a neural network's *inferred* depth from a single image (learned from shading/pixel patterns), not physically measured depth, so it is a supporting signal, not the primary anti-spoofing defense.

6.3 Motion-parallax depth verification (primary anti-spoofing signal)

This reuses the head-turn the student is already performing for the liveness challenge, at no extra cost to the user experience, and provides genuine measured depth rather than an estimate.

Principle: when a real 3D face turns, landmark points closer to the camera (nose tip) shift across the frame *faster* than points farther away (ears) — the same parallax effect that gives human stereo vision its sense of depth. A flat photo or screen replay has **no parallax**: every point shifts by the same amount regardless of orientation, because it is a rigid flat plane, not a 3D object.

```
// Pseudocode - parallax check across the turn challenge
List<LandmarkFrame> frames = capturedDuringChallenge; // ~10 fps over 4s

double noseDelta = distance(frames.first.noseTip, frames.last.noseTip);
double earDelta  = distance(frames.first.earTip, frames.last.earTip);

double parallaxRatio = noseDelta / earDelta;

// Real 3D face: nose (closer to camera) moves noticeably more than ear (farther)
// Flat photo/screen: ratio approaches 1.0 (uniform movement)
bool isLive = parallaxRatio > 1.3; // threshold tuned empirically per Section 9
```

Faking correct parallax without an actual 3D object in front of the camera is significantly harder than defeating a static photo check — this is the strongest single anti-spoofing signal available without dedicated depth hardware.

6.4 What this cannot defend against

A live, real-time AI deepfake video call feeding the camera could, in principle, still defeat challenge-response and parallax checks. This is treated as a low-probability threat for a student attendance context and is backstopped with occasional random manual spot checks by staff rather than further engineering effort, which would add friction disproportionate to the risk.

7. Face identity matching

Cosine similarity is computed between the live embedding and the stored (encrypted, decrypted server-side only for the comparison) reference embedding.

$\text{similarity}(A, B) = (A \cdot B) / (||A|| \times ||B||) \rightarrow \text{accept if } \geq \text{threshold (Section 9)}$

8. Geofencing — exact implementation

8.1 GPS inside a real campus polygon

```
-- PostGIS: precise polygon containment, not a naive radius circle
SELECT ST_Contains(
    boundary::geometry,
    ST_SetSRID(ST_MakePoint(:device_lng, :device_lat), 4326)
) AS inside_campus
FROM campuses WHERE id = :campus_id;
```

- Reject if reported GPS accuracy > 30m (ask app to retry for a better fix rather than accept a bad one).
- Reject immediately if the device reports a mock-location flag (Android `isFromMockProvider`) or matches a known GPS-spoofing app signature.

8.2 WiFi BSSID match

```
final connectedBSSID = await NetworkInfo().getWifiBSSID();
final isOnCampusWifi = campus.wifiBssids.contains(connectedBSSID);
```

The phone's connected access point MAC address is cross-checked against a whitelist of campus router BSSIDs, collected once during setup.

8.3 Decision rule

Approve only if: GPS-inside-polygon **AND** (WiFi-matched **OR** GPS-accuracy $\leq 10\text{m}$).
This lets students without WiFi still mark attendance if their GPS fix is precise, while requiring the stronger signal whenever GPS is loose.

8.4 Optional: BLE beacons for room-level precision

For institutions that need classroom-level (not just campus-level) accuracy, BLE beacons (~\$20-30 each) installed per room give a much tighter radius than GPS or WiFi can achieve indoors.

9. Threshold calibration — don't guess, measure

Before launch, run a validation pass with 50-100 real students: genuine attempts (same student) and impostor attempts (different student's photo/video/mask) against the live system. Plot False Accept Rate against False Reject Rate across a range of thresholds and pick the operating point where FAR is close to zero — a false accept (wrong person credited as present) is a worse outcome for an attendance system than an occasional false reject (real student has to retry).

Parameter	Starting value	Must be validated on your own population
Face match cosine similarity	0.65 - 0.70	Yes — lighting, camera quality, and demographics all shift this
Head yaw angle (liveness)	$\geq 20^\circ$	Minor tuning only
Parallax ratio	> 1.3	Yes — tune against real spoof attempts, not assumptions
GPS accuracy cutoff	$\leq 30\text{m}$ (reject above)	Test on your actual campus buildings

10. Database schema

```
CREATE EXTENSION IF NOT EXISTS postgis;
CREATE EXTENSION IF NOT EXISTS pgcrypto;

CREATE TABLE students (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  roll_number VARCHAR(20) UNIQUE NOT NULL,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  face_embedding BYTEA,
  embedding_enrolled_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE TABLE campuses (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name VARCHAR(100) NOT NULL,
  boundary GEOGRAPHY(POLYGON, 4326) NOT NULL,
  wifi_bssids TEXT[] NOT NULL
);

CREATE TABLE classes (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  campus_id UUID REFERENCES campuses(id),
  name VARCHAR(100) NOT NULL,
  start_time TIME NOT NULL,
```

```
end_time TIME NOT NULL,
days_of_week INT[] NOT NULL,
room_beacon_uuid VARCHAR(64)
);

CREATE TABLE attendance_records (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  student_id UUID REFERENCES students(id),
  class_id UUID REFERENCES classes(id),
  marked_at TIMESTAMPTZ DEFAULT now(),
  face_match_score FLOAT NOT NULL,
  liveness_passed BOOLEAN NOT NULL,
  liveness_action VARCHAR(20) NOT NULL,
  parallax_ratio FLOAT,
  device_lat DOUBLE PRECISION NOT NULL,
  device_lng DOUBLE PRECISION NOT NULL,
  gps_accuracy_m FLOAT,
  wifi_bssid_matched VARCHAR(20),
  mock_location_flag BOOLEAN DEFAULT false,
  device_id VARCHAR(100) NOT NULL,
  status VARCHAR(20) NOT NULL,
  UNIQUE (student_id, class_id, marked_at)
);

CREATE INDEX idx_attendance_class_time ON attendance_records (class_id, marked_at);
CREATE INDEX idx_campus_boundary ON campuses USING GIST (boundary);
```

11. Backend API design

Endpoint	Purpose
POST /auth/login	Student login (roll number + password)
POST /enrollment/submit	Upload 5 face photos for enrollment
GET /enrollment/status	Pending / approved / rejected
POST /attendance/mark	Core verification endpoint — see request shape below
GET /attendance/history/:studentId	Student's own attendance record
GET /admin/attendance/:classId	Staff view, flagged records highlighted
POST /admin/enrollment/:id/approve	Staff approves an enrollment
POST /admin/campus	Define campus polygon + WiFi whitelist

POST /attendance/mark — request body

```
{
  "classId": "uuid",
  "faceEmbedding": [0.021, -0.113, ...],
  "livenessAction": "turn_right",
  "livenessPassed": true,
  "parallaxRatio": 1.42,
  "deviceLat": 12.3052,
  "deviceLng": 76.6552,
  "gpsAccuracy": 8.2,
  "wifiBssid": "AA:BB:CC:DD:EE:FF",
  "mockLocationFlag": false,
  "deviceId": "hashed-device-fingerprint"
}
```

Server-side validation sequence (reject at first failure, log the reason)

1. Is `now()` within the class's valid time window (server clock, never device clock)?
2. Cosine similarity between submitted and stored embedding $\geq$ threshold?
3. Was liveness passed, including parallax ratio above threshold, within the time limit?
4. PostGIS polygon check + WiFi BSSID check pass?
5. `mockLocationFlag` is false?
6. Has this `deviceId` already marked attendance for 3+ different students today (buddy-punching detector)? If so, flag for manual review instead of auto-rejecting.
7. Insert record, return success with the reasons logged either way.

12. Security & compliance

- Face embeddings encrypted at rest (AES-256, KMS-managed keys) and in transit (TLS 1.3).
- Raw enrollment photos are **not retained** beyond admin approval — only the encrypted embedding persists.
- **India — Digital Personal Data Protection Act, 2023 (DPDP Act):** biometric data collection requires explicit informed consent, a clearly stated purpose, and a defined retention/deletion policy. Parental consent is required for minors. Get institutional legal sign-off before collecting biometric data from any student, particularly anyone under 18.
- Rate-limit `/attendance/mark` per device via Redis to block automated abuse.
- Full audit log on every record — match score, liveness action, parallax ratio, and location data — so any disputed record can be manually reviewed rather than blindly trusted.

13. Testing strategy

- **Unit tests:** threshold logic, geofence polygon math, time-window edge cases (midnight-crossing classes).
- **Adversarial testing before launch:** attempt to beat the system with printed photos, phone-replayed video, GPS-spoofing apps, and WiFi MAC spoofing — confirm each is caught.
- **Device diversity testing:** at least 3–4 Android price tiers plus iOS, since camera quality varies enormously.
- **Load testing:** simulate an entire class (e.g. 60 students) marking attendance within the same 2-minute window.

14. Development roadmap

Phase	Scope	Duration
1	Backend core: auth, DB schema, class/campus admin panel	2–3 weeks
2	Mobile app: enrollment flow + admin approval	2 weeks
3	Face match + liveness + parallax integration	3–4 weeks (most technically involved)
4	Geofencing (GPS + WiFi) + time window	1–2 weeks
5	Threshold calibration with real student data	1 week
6	Adversarial + load testing, fixes	2 weeks
7	Pilot with one class/department, monitor flagged records	2–4 weeks
8	Full rollout	—

Total to a solid pilot: roughly 3.5–4 months for a small team (1–2 mobile developers, 1 backend developer).

15. Cost estimate

- On-device ML inference: **\$0** — no per-verification API cost, since matching and liveness both run on the phone.
- Backend hosting (Supabase Pro or small AWS setup): **\$25–100/month** depending on scale.
- Push notifications (Firebase): free tier covers most institutions.
- BLE beacons (optional, room-level precision): \$20–30/beacon, one-time.

16. Known limitations

- Cannot fully defend against a live, real-time deepfake video-call attack — mitigated with occasional manual spot checks, not further automated engineering.
- Indoor / multi-story GPS accuracy can remain poor even with WiFi cross-checks — BLE beacons are the fix for large lecture halls with weak signal.
- Face matching accuracy is affected by significant appearance changes (new glasses, facial hair, injury) — a manual staff override path is required, not just automated rejection.
- Requires every student to own a smartphone with a working front camera and GPS — a fallback manual-attendance path is needed for students without one.

17. Appendix — full requirements checklist

#	Requirement
1	Flutter mobile app: enrollment, login, liveness challenge UI, attendance history
2	On-device ML: ML Kit face detection, MediaPipe Face Mesh, MobileFaceNet TFLite embeddings
3	Parallax computation module tied to the head-turn challenge frames
4	NestJS backend with the 8 endpoints listed in Section 11
5	PostgreSQL + PostGIS schema exactly as specified in Section 10
6	Admin panel: campus polygon drawing tool, WiFi BSSID whitelist entry, enrollment approval queue, flagged-record review
7	Redis-backed rate limiting on the attendance endpoint
8	AES-256 encryption for stored embeddings, TLS 1.3 in transit
9	Server-authoritative timestamps and geofence checks — never trust client-reported time or location alone
10	Consent flow compliant with India's DPDP Act, including parental consent path for minors